Universidad Autónoma de Baja California

Facultad de Ciencias Quimicas e Ingenieria

Ingeniería en Computación

Navarrete Pérez Andrés - 1284707

Proyecto de carrera

Taller 4



Taller 4. Alternativas técnicas y viabilidad

ChiTito Dispenser será un sistema inteligente para alimentación y suministro de agua para mascotas. Permitirá identificar a la mascota mediante RFID, configurar la cantidad de alimento desde una aplicación, dispensar automáticamente la porción y registrar la cantidad realmente servida.

También permitirá conocer aproximadamente el nivel de alimento y agua, mantener el agua en circulación y monitorear cambios en su condición. Las funciones críticas se ejecutarán localmente para que el dispensado, el acceso RFID y la protección de la bomba no dependan de Internet.

Objetivo del análisis

Comparar dos alternativas técnicas completas y seleccionar la solución mediante una matriz de decisión ponderada con criterios técnicos, económicos, de seguridad y de desarrollo.

Componentes funcionales comunes

Componente	Interfaz	Función
RC522	SPI	Identificación RFID
VL53L1X	I2C	Nivel de alimento
Celda 5 kg + HX711	GPIO	Peso de porción
2 x MG90S	PWM	Dosificación y acceso
A02YYUW	UART	Nivel de agua
SEN0244	ADC	TDS
SEN0189	ADC	Turbidez
Bomba 5 V + AO3400A	GPIO	Recirculación de agua

Alternativa técnica 1 - ESP32 + MQTT + Mosquitto + Laravel + MySQL

Sistema embebido

ESP32 DevKit V1. Integra Wi-Fi de 2.4 GHz y periféricos SPI, I2C, UART, PWM, GPIO y ADC suficientes para centralizar los sensores y actuadores del prototipo.

Sensores y actuadores principales

Se utilizarán los componentes definidos anteriormente. Los servomotores y la bomba tendrán alimentación externa de 5 V y compartirán tierra con el ESP32.

Comunicación

Wi-Fi de 2.4 GHz y MQTT. El ESP32 publicará telemetría y eventos y recibirá configuraciones y comandos mediante tópicos. Para Internet se plantea MQTT sobre TLS.

Backend y servicio en la nube

Eclipse Mosquitto funcionará como broker MQTT y Laravel como backend/API. Mosquitto, Laravel y MySQL podrán alojarse en un mismo VPS básico.

Base de datos

MySQL para usuarios, mascotas, configuraciones, historial de porciones, alertas y estado del dispositivo.

Frontend

Vue 3 como Progressive Web App (PWA), adaptable a teléfono y computadora.

Seguridad

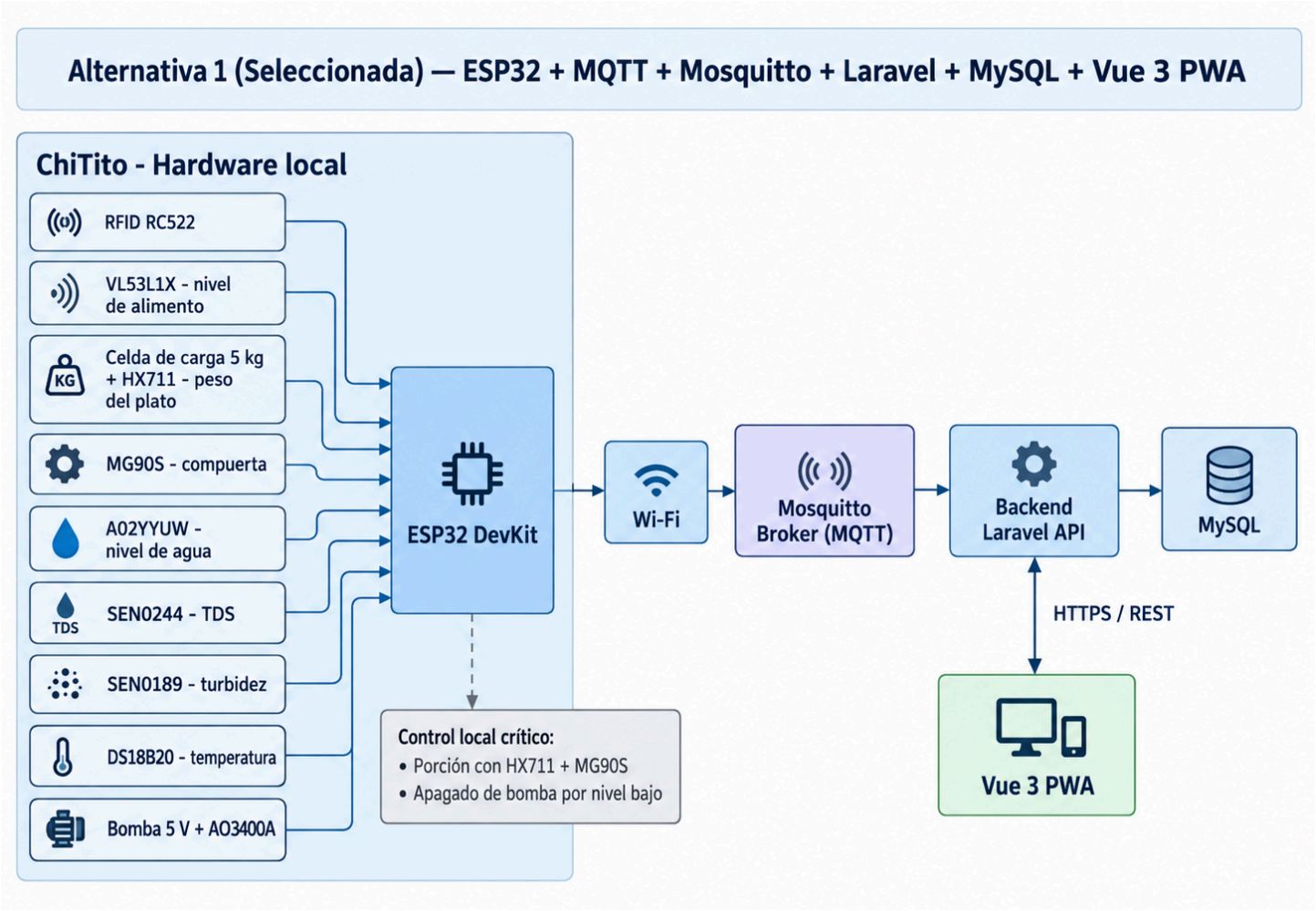
HTTPS para frontend/API, autenticación, hashing de contraseñas, validación, roles y protección de endpoints. En MQTT: TLS, credenciales por dispositivo y ACL para restringir tópicos.

Costo, complejidad y tiempo

Costo físico preliminar cercano a $2,500 MXN. La complejidad es media-alta por la integración de firmware, MQTT, broker, backend, base de datos y frontend. Tiempo estimado: 8 a 10 semanas.

Semana	Actividad
1	Diseño eléctrico y mecánico
2	Pruebas de sensores
3	HX711 y celda
4	MG90S y dispensado
5	Sistema de agua
6	RFID y monitoreo
7	MQTT + Mosquitto
8	Laravel + MySQL
9	Vue 3 PWA
10	Integración y calibración

Diagrama de bloques - Alternativa 1



Alternativa técnica 2 - Raspberry Pi Pico W + HTTPS/REST + FastAPI + PostgreSQL

Sistema embebido

Raspberry Pi Pico W basada en RP2040. Integra Wi-Fi de 2.4 GHz, 26 GPIO multifunción, tres entradas ADC, SPI, I2C, UART y PWM. Puede controlar el mismo conjunto funcional, aunque dos de sus tres entradas ADC quedarían ocupadas por TDS y turbidez.

Sensores y actuadores principales

Se mantienen los mismos componentes para comparar soluciones equivalentes: RC522 por SPI, VL53L1X por I2C, A02YYUW por UART, HX711 por GPIO, dos MG90S por PWM, SEN0244 y SEN0189 por ADC y bomba mediante AO3400A.

Comunicación

Wi-Fi de 2.4 GHz con HTTPS y API REST. La Pico W enviaría lecturas y eventos mediante solicitudes HTTP y consultaría cambios de configuración. Se elimina el broker MQTT, pero se requiere un mecanismo de consulta o sincronización para recibir cambios remotos.

Backend y servicio en la nube

FastAPI como backend REST desplegado en un servicio cloud o VPS. La API recibiría mediciones, validaría dispositivos y expondría endpoints para la PWA.

Base de datos

PostgreSQL para usuarios, mascotas, configuración, porciones, telemetría y alertas.

Frontend

Vue 3 PWA, manteniendo la misma experiencia funcional que la alternativa 1.

Seguridad

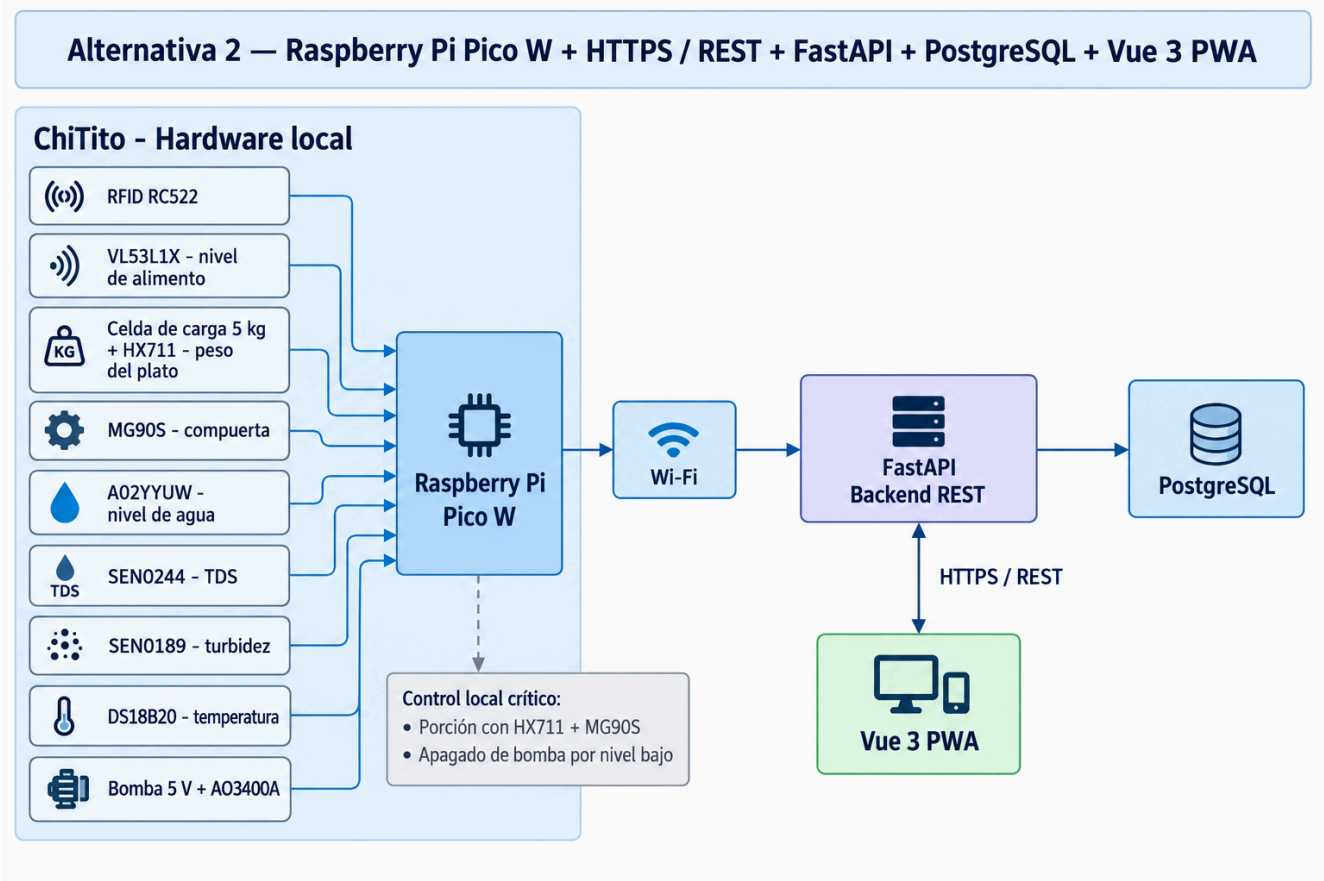
HTTPS/TLS, autenticación de usuarios, tokens para la aplicación, credencial individual del dispositivo, validación de solicitudes, autorización de endpoints y limitación de tasa.

Costo, complejidad y tiempo

Costo físico aproximado de $2,450 a $2,600 MXN. La complejidad es media: se elimina Mosquitto, pero se implementan la lógica REST, autenticación del dispositivo y sincronización. Tiempo estimado: 9 a 11 semanas.

Semana	Actividad
1	Diseño eléctrico y mecánico
2	Sensores con Pico W
3	HX711 y ADC
4	Servos y dispensado
5	Sistema de agua
6	RFID y monitoreo
7	HTTPS + API REST
8	FastAPI + PostgreSQL
9	Vue 3 PWA
10	Integración
11	Calibración y pruebas

Diagrama de bloques - Alternativa 2



Comparación de alternativas

Aspecto	Alternativa 1	Alternativa 2
Sistema embebido	ESP32 DevKit V1	Raspberry Pi Pico W
Sensores y actuadores	RC522, VL53L1X, HX711, 2 MG90S, A02YYUW, TDS, turbidez y bomba	Mismos sensores y actuadores
Comunicación	Wi-Fi + MQTT/TLS	Wi-Fi + HTTPS/REST
Backend/nube	Mosquitto + Laravel en VPS	FastAPI REST en cloud/VPS
Base de datos	MySQL	PostgreSQL
Frontend	Vue 3 PWA	Vue 3 PWA
Costo	≈ $2,500 MXN	≈ $2,450-$2,600 MXN
Complejidad	Media-alta; arquitectura IoT por capas	Media; REST, con consulta/sincronización
Tiempo	8-10 semanas	9-11 semanas
Seguridad	TLS, ACL MQTT, credenciales y HTTPS	HTTPS, tokens, credenciales y rate limiting

Matriz de decisión ponderada

Escala: 1 = muy desfavorable, 2 = desfavorable, 3 = aceptable, 4 = favorable y 5 = muy favorable. Puntaje ponderado = peso × calificación / 5.

Criterio	Peso	A1 Calif.	A1 Pond.	A2 Calif.	A2 Pond.
Integración con sensores y actuadores	20%	5	20.0	4	16.0
Adecuación de comunicación a IoT	18%	5	18.0	3	10.8
Control local y operación sin Internet	15%	5	15.0	5	15.0
Seguridad	12%	4	9.6	4	9.6
Costo total	10%	4	8.0	4	8.0
Complejidad de integración	10%	4	8.0	3	6.0
Tiempo de desarrollo	8%	4	6.4	3	4.8
Escalabilidad y mantenimiento	7%	5	7.0	4	5.6
TOTAL	100%		92.0/100		75.8/100

Justificación de la alternativa elegida

Se selecciona la alternativa 1 con ESP32, MQTT, Mosquitto, Laravel, MySQL y Vue 3 PWA. La decisión no se basa únicamente en que sea una tecnología conocida. La matriz favorece esta solución por la integración directa del ESP32 con los periféricos requeridos, la comunicación publicación/suscripción de MQTT y la recepción de comandos sin consultas periódicas al servidor.

MQTT permite publicar eventos y telemetría y mantener al dispositivo suscrito a configuraciones o comandos. Esto es adecuado para un dispensador IoT que debe reaccionar localmente y sincronizarse con una aplicación. La alternativa 2 también es viable, pero obtiene menor puntuación por la sincronización mediante REST, la adaptación adicional del firmware y la menor disponibilidad de entradas ADC.

Lista preliminar de materiales - Alternativa seleccionada

Componente	Modelo	Cant.	Función	Costo MXN
ESP32 DevKit V1	ESP32	1	Control principal	220
Lector RFID	RC522	1	Identificación	75
Tags RFID	MIFARE 13.56 MHz	2	Identificador de mascota	50
Sensor alimento	VL53L1X	1	Nivel de alimento	150
Celda + convertidor	5 kg + HX711	1	Peso de porción	95
Servomotores	MG90S 180°	2	Dosificación y acceso	140
Sensor nivel agua	A02YYUW / SEN0311	1	Nivel de agua	320
Sensor TDS	DFRobot SEN0244	1	Condición del agua	240
Sensor turbidez	DFRobot SEN0189	1	Condición del agua	210
Bomba	5 V, 80-100 L/h	1	Recirculación	45
MOSFET/protección	AO3400A + diodo/resistencias	1	Control de bomba	30
Fuente	5 V / 5 A	1	Alimentación	260
Depósitos y plato	Grado alimenticio	1	Estructura	300
Electrónica auxiliar	PCB, cables, conectores	1	Integración	220
Mecánica/impresión 3D	Compuertas y soportes	1	Prototipo	400

Total preliminar aproximado: $2,755 MXN. El valor es una estimación y puede variar según proveedor, envío, disponibilidad y piezas que ya se tengan.

Estimación preliminar de costos recurrentes de nube o comunicación

Para la alternativa seleccionada se propone un VPS básico con 1 GB de RAM. DigitalOcean publica un precio de US$6 al mes para un Basic Droplet de 1 GiB, 1 vCPU y 25 GiB SSD. Para un prototipo de bajo tráfico puede alojar Mosquitto, Laravel y MySQL si se configura de forma adecuada. La conexión Wi-Fi utilizará la red existente, por lo que no se considera una cuota celular adicional.

Como referencia para la alternativa 2, PostgreSQL podría mantenerse en el mismo VPS o emplear un servicio administrado. Supabase ofrece un plan gratuito con límites de uso y planes pagados cuando el proyecto requiera mayor capacidad.

Viabilidad técnica

La alternativa 1 es técnicamente viable porque el ESP32 dispone de Wi-Fi y periféricos SPI, I2C, UART, PWM, GPIO y ADC. El control de porción se realizará con retroalimentación: el MG90S abre la compuerta, el HX711 mide continuamente el peso y el ESP32 cierra al alcanzar la cantidad configurada. El RFID autoriza el acceso al plato y el A02YYUW protege la bomba cuando el nivel de agua es bajo.

Las funciones críticas permanecerán locales. Una caída del VPS o de Internet afectará el monitoreo remoto y la sincronización, pero no deberá impedir una operación básica segura.

Viabilidad económica

El costo del prototipo se mantiene dentro de un rango razonable para un proyecto universitario y la mayor parte corresponde a sensores, estructura y mecánica. El costo recurrente principal sería el VPS, aproximadamente US$6 mensuales en la configuración de referencia.

Viabilidad operativa

El usuario operará el sistema desde una PWA para seleccionar mascota, configurar porciones, revisar niveles y consultar alertas. El mantenimiento esperado incluye limpieza de depósitos, revisión de la bomba, calibración de la celda de carga y limpieza de los sensores de agua.

Viabilidad temporal

El plan de 10 semanas de la alternativa seleccionada permite desarrollar primero el control local y después integrar conectividad, backend y frontend. Esta secuencia reduce el riesgo de depender de la nube antes de validar el funcionamiento físico.

Restricciones y riesgos técnicos principales

Riesgo o restricción	Consecuencia	Mitigación
Corto alcance/orientación del RC522	Lecturas intermitentes	Colocar el lector cerca del acceso y probar la posición del tag
Dos mascotas simultáneas	Acceso no deseado	Diseño de acceso individual estrecho
Cierre con mascota dentro	Riesgo físico	Cierre lento, retardo y diseño sin atrapamiento
Atasco de compuerta	Porción incompleta	Diseño inclinado, timeout y pruebas
Croquetas continúan cayendo	Exceso de alimento	Cierre progresivo usando HX711
Deriva HX711	Peso incorrecto	Calibración periódica
Variación VL53L1X	Nivel irregular	Promedio o mediana de lecturas
Nivel de agua bajo	Bomba en seco	Apagado local por A02YYUW
Sensores de agua sucios	Lecturas incorrectas	Mantenimiento programado
Ruido de servos/bomba	Reinicios	Fuente 5 V/5 A, capacitores y separación de potencia
Caída de MQTT/VPS	Sin monitoreo remoto	Control local y reconexión automática
Acceso MQTT no autorizado	Control indebido	TLS, credenciales y ACL
Lecturas TDS/turbidez mal interpretadas	Falsa idea de potabilidad	Mostrar condición del agua, no certificación sanitaria

Actualización del alcance aprobado

• ESP32 DevKit V1 como sistema embebido seleccionado.

• Identificación por RFID con un tag por mascota.

• Dosificación con MG90S y medición de porción mediante celda de carga + HX711.

• Control de acceso individual al plato con un segundo MG90S.

• VL53L1X para nivel de alimento y A02YYUW para nivel de agua.

• Recirculación con bomba de 5 V y AO3400A.

• Monitoreo de TDS y turbidez como indicadores de condición del agua.

• Wi-Fi + MQTT + Mosquitto + Laravel + MySQL + Vue 3 PWA.

• Historial de porciones, configuración remota y alertas.

Fuera del alcance inicial

Se mantienen fuera del alcance: reconocimiento por cámara, inteligencia artificial, reconocimiento facial, análisis microbiológico, certificación de potabilidad, medición exacta del agua consumida por mascota, cambio automático del agua y aplicación nativa independiente para Android/iOS.

Conclusión

Las dos alternativas pueden implementar ChiTito Smart Dispenser, pero la matriz ponderada respalda mantener la alternativa 1. El ESP32 ofrece las interfaces necesarias y MQTT se adapta mejor al intercambio bidireccional de eventos y comandos de un sistema IoT. La propuesta conserva el control crítico en el dispositivo, mientras que el VPS se utiliza para mensajería, almacenamiento, administración y acceso remoto.

Referencias

[1] Espressif Systems, “ESP32 Series Datasheet,” versión 5.2, 2025.

[2] Raspberry Pi Ltd., “Raspberry Pi Pico W Datasheet,” 2025.

[3] DigitalOcean, “Droplet Pricing,” consultado en septiembre de 2026.

[4] Supabase, “About billing on Supabase,” consultado en septiembre de 2026.

[5] Eclipse Foundation, “Eclipse Mosquitto - An open source MQTT broker.”

[6] Laravel, “Laravel Documentation.”